第二阶段：正数、负数和边界值测试报告

工大喵喵队 2026 年 11 月

一、测试设计

第二阶段 2.3 要求测三类：正数点积、负数点积、边界值。

我们没有拆成三个小程序，而是写了一个 `software/main.c`，循环 1,030 个 test case，每个 case 生成一组 A/B，再用软件 oracle 算期望值。每个 case 依次调用 5 个实现：

变体	说明
`vdot_custom`	正常 vdot，vd=v3
`vdot_baseline`	标准 RVV `vwmul.vv + vwredsum.vs`
`vdot_alias1`	vd=vs1=v1
`vdot_alias2`	vd=vs2=v2
`vdot_v0`	vd=v0

每次调用前把 `out[0..3]`填 `0x55555555`，调用后检查：

1.`out[0] == oracle`； 2.`out[1] == out[2] == out[3] == 0`，也就是高 96 位清零。

任何一个检查失败，程序返回 1，不会走到 GOOD TRAP。a0 版把 trap 字的 rs1 改成 a0（`.word 0x0005006b`），只有 1,030 个 case × 5 个变体都满足，才会触发 a0=0 的 trap 字，打印 `HIT GOOD TRAP`。

二、正数点积

case	A	B	期望
0	全 127	全 127	129,032
4	−127/127 交替	全 127	0
5	前 8 字节 0，高 8 字节 −128	同	0
6–1029	随机 int8	随机 int8	oracle

RTL 单元测试里还有 `positive` 定向用例和 65,536 个全字节对穷举。

整机 a0 回归里 case 0 的期望是 127×127×8 = 129,032，实际输出和 oracle 一致，`out[1..3]`都是 0。随机 case 也包含在这轮 a0 回归里。

日志在 `results/t5b-clean-ref/vdot-a0.log`，最后是：

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 841,260, cycleCnt = 790,553, IPC = 1.064141
```

三、负数点积

case	A	B	期望
1	全 −128	全 −128	131,072
2	全 −128	全 127	−130,048
随机	随机 int8	随机 int8	oracle

RTL 单元测试里还有 `negative`（−1..−8 × 2）、`max_negative`（−128×127）和随机负组合。

负数点积最重要的是符号扩展：`0x80` 解释成 −128，不是 +128；`0xFF` 解释成 −1，不是 255。数据通路用 `asSInt` 做有符号乘法，NEMU helper 里用：

```
int32_t a = vreg_b(id_src->reg, i);
int32_t b = vreg_b(id_src2->reg, i);
if (a >= 128) a -= 256;
if (b >= 128) b -= 256;
sum += a * b;
```

case 1 结果是 131,072，case 2 是 −130,048，随机负数组合也全部和 oracle 一致。

四、边界值

边界	输入	期望
零	0 × −128	0
最大正积	−128 × −128	131,072
最大负数	−128 × 127	−130,048
最大 int8 正	127 × 127	129,032

边界	输入	期望
抵消	127, −127 交替	0
高位忽略	高 8 字节 −128	只算前 8 字节

RTL 单元测试里有 65,536 个全字节对穷举（256×256），把 int8 乘法的符号组合都过了一遍。

高位忽略用 case 5 检查：前 8 字节设 0，高 8 字节设 −128，期望 0。如果实现错误地读了高 8 字节，结果会变成正数，测试就会失败。实际结果是 0。

五、整机回归

1,030 个 test case × 5 个实现变体，a0 版为 GOOD TRAP，a0=0（退出码 0）：

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 841,260, cycleCnt = 790,553, IPC = 1.064141
```

历史 `vdot-regression.log` 用的是 x0 trap 字，只能说明运行至结束、无 mismatch；这里的结果以 `vdot-a0.log` 为准。

· 无 difftest mismatch；

· 无 CRITICAL ERROR；

· 无断言失败。

841,260 是整个测试程序的指令数，不是 841,260 次 vdot。按 5,150 次点积调用估算，平均每次调用约 163 条指令，包含循环、装载、比较和函数调用。

六、RTL 单元测试

独立 SV 原型 `rtl/vdot8.sv` 和 testbench `tests/tb_vdot8.sv` 用 69,656 个 oracle 向量做穷举：

· 24 个定向用例：正/负/零/极值/抵消/高位忽略/lane 0–15；

· 65,536 个全字节对穷举：256×256；

· 4,096 个随机向量。

testbench 还检查满吞吐、气泡、随机背压、输出保持、两级流水填满后 flush、有未完成事务时 reset，以及 4 个取消事务。

结果：

```
PASS vectors=69656 checked=69657 cancelled=4 cycles=108440
```

这两层验证对象不同：独立 SV testbench 验证数据通路，整机 difftest 验证 Scala 集成版。

七、异常和别名

software/exception_check.S 的 a0 版做了三次 trap：

1.VL=0 执行 vdot → 第 1 次 trap； 2.标准非法指令 0x0000007B → 第 2 次 trap； 3.再次 VL=0 执行 vdot → 第 3 次 trap。

每次都检查 mcause=2、mepc 指向对应指令、vdot 的 mtval=0、标准非法指令的 mtval=0x7B、v3 仍是 0xFF、mret 后能继续执行。日志是 results/t5b-clean-ref/exception-check-a0.log，HIT GOOD TRAP、a0=0，274 条指令、2,865 周期，无 mismatch。

8 种非法配置的 mtval 由 software/mtval_probe.c 逐例打印，证据是 results/t2-mtval-probe/mtval-probe.log：

配置	mcause	mtval	发现阶段
VL=0/1/7/9	2	0x0	执行阶段（VIMacU dotIllegal）
SEW=16/LMUL=2/vstart=1	2	0x0220a18b	译码阶段（DecodeUnit，TrapInstMod 填指令字）
标准非法 0x7b	2	0x7b	常规非法指令

这次 probe 是`--no-diff`跑的，直接打印每例`mcause/mepc/mtval`，不是 difftest。带 difftest 的 A/B 只对照了 SEW=16：修复前`right=0`、`wrong=0x220a18b`，修复后无 mismatch。LMUL=2 和 vstart=1 的 mtval 还没有在带 difftest 的运行里逐例比对过。

别名方面，`vdot_alias1`、`vdot_alias2`、`vdot_v0`各跑 1,030 个 case，包含在 a0 回归中，a0=0。

高位独立性用 4 组高半图案：先 VL=16 全宽加载并回存，确认高 8 字节确实进了寄存器，再用 VL=8 执行 vdot。程序输出`echo16=ok`、`low=-168`、`upper96=0,0,0`，日志`results/t3-highbits-backtoback-v2/highbits-v2.log`。背靠背测试是三条紧邻的 vdot，结果−168/204/−4，日志`results/t3-highbits-backtoback-v2/backtoback.log`。这两组是程序自己打印`PASS`、exit 0，不是 a0 回归。

八、结果汇总

测试	结果	指令数	周期数
RTL 单元（69,656 向量）	独立 testbench 输出`PASS`	—	108,440
vdot 功能（1,030×5）	a0 版 GOOD TRAP，a0=0	841,260	790,553
vadd 标准回归	a0 版 GOOD TRAP，a0=0	23,876	16,870
VL=0 异常	a0 版 GOOD TRAP，a0=0	274	2,865
8 种非法配置	mtval 两类，逐例打印（`--no-diff`）	—	—
高位/背靠背	程序输出`PASS`，exit 0	—	—

九、没覆盖的地方

· 异常覆盖 M 模式 8 种非法配置；S/VS 委托、RNMI、NMIE=0 早退路径未覆盖；

· `vm=0`等其他非法配置没有进 8 例矩阵；

· 8 例 mtval 是`--no-diff`直接打印；带 difftest 的 A/B 只对照了 SEW=16 和 VL=0；

· 没有覆盖率数字，69,656 向量和 1,030 用例是实测覆盖，不是形式化证明；

· 独立 SV 原型和 Scala 集成版不是同一份代码，不能互相替代结论。